CS39006 – Database Management Systems Laboratory

Project Proposal

Hybrid Trading Database System: High-QPS Order Matching, Temporal Analytics, and Quantum-Assisted Arbitrage Detection

Course:	CS39006 – Database Management Systems
Semester:	Spring 2026
Submission Date:	March 11, 2026
Final Demo Due:	April 15, 2026

Group Members

S.No.	Name	Roll Number
1	Ashutosh Sharma	23CS10005
2	Sujal Anil Kaware	23CS30056
3	Parag Mahadeo Chimankar	23CS10049
4	Kshetrimayum Abo	23CS30029
5	Kartik Pandey	23CS30026

Hybrid Trading Database System: High-QPS Order Matching, Temporal Analytics, and Quantum-Assisted Arbitrage Detection

Course Guideline Coverage: This project spans multiple categories from the provided guidelines, including **Guideline #2** (High QPS Systems), **Guideline #6** (Temporal Databases), **Guideline #10** (NoSQL/Graph Databases), **Guideline #28** (Database Security), and **Guideline #29** (Performance Analysis and Benchmarking).

Abstract

This project presents the design and full implementation of a **Hybrid Trading Database System** that integrates five specialized database paradigms into a single, production-inspired infrastructure. The system consists of: (1) a **High-QPS In-Memory Limit Order Book (LOB)** engine implementing the LMAX Disruptor pattern for microsecond-level order matching with lock-free concurrency; (2) a **TimescaleDB Temporal Database** for persistent tick-data ingestion with hierarchical, automatically-refreshed OHLCV candlestick continuous aggregates and SQL-computed live technical indicators; (3) an **Apache AGE Graph Database** layer modeling multi-currency exchange rates as a directed weighted graph with sub-second live edge-weight updates; (4) a **Quantum-Assisted Arbitrage Detection Engine** using Grover's Search Algorithm (IBM Qiskit) to detect profitable triangular arbitrage cycles in $O(\sqrt{N})$ circuit iterations, achieving a provable quadratic speedup over the classical Bellman-Ford $O(N)$ baseline; and (5) a **Security and Observability Layer** featuring a SQL injection prevention middleware (AST-based query parsing), rate-limited DoS protection backed by Redis, and a live Grafana dashboard visualizing order book depth, candlestick charts, and quantum arbitrage signal feeds. The entire system will be benchmarked using Siege and custom multithreaded HTTP clients to produce detailed QPS throughput and latency reports.

System Architecture Overview

The system is structured as a five-layer pipeline where data flows from market ingestion through matching, persistence, graph analysis, and quantum processing:

[LAYER 0] Market Data Ingestion WebSocket clients (Binance / Alpha Vantage API) → Apache Kafka Topic `raw_orders`
[LAYER 1] High-QPS Limit Order Book Engine (Module 1) Red-Black Tree (price levels) + FIFO Doubly Linked List (per level) LMAX Disruptor Ring Buffer for inter-thread communication Target throughput: >100,000 orders/sec on commodity hardware
[LAYER 2] TimescaleDB Temporal Store (Module 2) Hypertable: `raw_ticks` partitioned by (time, symbol) Continuous Aggregates: 1m / 5m / 15m / 1h OHLCV candles Live SQL indicators: VWAP, SMA-20, Bollinger Bands, RSI
[LAYER 3] Graph Database Layer -- Apache AGE (Module 3) Nodes: Traded assets (BTC, ETH, USD, EUR, INR . . . 20+ nodes) Directed weighted edges: live best-bid exchange rates Background worker updates edge weights every 500ms
[LAYER 4] Quantum Arbitrage Detection Engine (Module 4) Grover's Algorithm (IBM Qiskit) encodes profitable 3-cycle constraint $O(\sqrt{N})$ search vs Bellman-Ford $O(N)$ classical baseline Writes detected opportunities to `arbitrage_signals` table
[LAYER 5] Security, Monitoring & Benchmarking (Module 5) SQL injection AST firewall + Redis-backed rate limiter Grafana dashboard: LOB depth, candlesticks, arbitrage signal feed Siege benchmarking suite: QPS stress tests under DDoS simulation

Module Descriptions

Module 1 – High-QPS Limit Order Book Matching Engine

The Limit Order Book (LOB) is the core transactional engine. It maintains all outstanding Buy and Sell orders sorted by price. Each **price level** is managed by a self-balancing **Red-Black Tree** with $O(\log M)$ insertion time (where M is the number of distinct price levels). At each price level, orders are stored in a **doubly linked list** in FIFO priority, enabling $O(1)$ execution of the best bid or ask.

To achieve maximum throughput without lock contention, the engine uses a simplified version of the **LMAX Disruptor pattern** – a fixed-size ring buffer that replaces traditional blocking queues. This enables near-zero-latency message passing between three threads: the **Inbound Thread** (receives orders), the **Matching Thread** (executes the LOB logic), and the **Persistence Thread** (batches executed trades to TimescaleDB).

API Operations: `PLACE`, `CANCEL`, `MODIFY`, `QUERY_DEPTH`

Benchmarking: Siege and a custom Python ThreadPool will be used to simulate

thousands of concurrent clients and measure peak QPS and 99th-percentile latency.

Module 2 – TimescaleDB Temporal Analytics Engine

All trade executions from the LOB engine are batch-inserted into a TimescaleDB **hypertable**, which automatically partitions data by time interval and asset symbol. This enables queries across millions of rows to complete in tens of milliseconds rather than seconds.

Continuous Aggregates automatically maintain pre-computed OHLCV (Open, High, Low, Close, Volume) candlestick data across multiple timeframes:

Listing 1: Continuous aggregate for 1-minute OHLCV candles

```sql
CREATE MATERIALIZED VIEW ohlcv_1m
WITH (timescaledb.continuous) AS
SELECT
    time_bucket('1 minute', ts)  AS bucket,
    symbol,
    FIRST(price, ts)             AS open,
    MAX(price)                   AS high,
    MIN(price)                   AS low,
    LAST(price, ts)              AS close,
    SUM(volume)                  AS volume
FROM raw_ticks
GROUP BY bucket, symbol;
```

Technical Indicators computed via SQL window functions without exporting data:

Listing 2: VWAP and 20-period SMA computed entirely in SQL

```sql
-- Volume Weighted Average Price (VWAP)
SELECT symbol,
       SUM(price * volume) / SUM(volume) AS vwap
FROM raw_ticks
WHERE ts > NOW() - INTERVAL '1 day'
GROUP BY symbol;

-- 20-Period Simple Moving Average
SELECT bucket, symbol, close,
       AVG(close) OVER (
           PARTITION BY symbol
           ORDER BY bucket
           ROWS BETWEEN 19 PRECEDING AND CURRENT ROW
       ) AS sma_20
FROM ohlcv_1m;
```

Module 3 – Graph Database Layer (Apache AGE)

The multi-currency exchange market is modeled as a **directed weighted graph** using **Apache AGE**, an open-source graph extension for PostgreSQL that supports Cypher-style graph queries natively.

- **Nodes**: Each traded asset (BTC, ETH, USD, EUR, INR, etc.) is a vertex.
- **Edges**: Each active trading pair is a directed edge. Edge weight = current best-bid exchange rate from the LOB (updated every 500ms by a background worker).

- **Classical Baseline**: Bellman-Ford on negative-log-transformed edge weights detects negative cycles (= arbitrage). This serves as the $O(N)$ classical benchmark.

Example graph query to find all paths from USD with depth 3:

Listing 3: Cypher query via Apache AGE to find 3-hop paths

```
SELECT * FROM cypher('fx_graph', $$
    MATCH p = (a:Asset {symbol: 'USD'})-[*3]->(a)
    WHERE reduce(rate=1.0, r IN relationships(p) | rate * r.bid) > 1.0
    RETURN p
$$) AS (path agtype);
```

Module 4 – Quantum Arbitrage Detection Engine (IBM Qiskit)

Detecting a profitable triangular arbitrage cycle among N currency pairs is an **unstructured search problem**. Grover's Algorithm provides a provable $O(\sqrt{N})$ speedup over classical $O(N)$ linear search for unstructured databases.

Implementation Pipeline:

1. A Python middleware queries the graph edge table every second to retrieve the latest exchange rate matrix.
2. The middleware encodes all possible 3-node cycle combinations as binary states in a quantum register.
3. A **Grover Oracle** circuit (in IBM Qiskit) marks states where the product of three exchange rates satisfies:
$$r_{AB} \times r_{BC} \times r_{CA} > 1.0$$
4. The **Diffuser operator** performs amplitude amplification via phase inversion around the mean, increasing the probability of marked (profitable) states after $\approx \frac{\pi}{4}\sqrt{N}$ iterations.
5. Measurement collapses the quantum state to the most probable profitable cycle.
6. The detected arbitrage path (e.g., USD $\rightarrow$ BTC $\rightarrow$ ETH $\rightarrow$ USD), profit percentage, and timestamp are written to the `arbitrage_signals` table in PostgreSQL.

Benchmarking: Wall-clock time of Qiskit simulation vs. Bellman-Ford will be measured for graph sizes $N \in \{8, 16, 32, 64\}$ nodes and plotted to visualize the quadratic speedup advantage.

Module 5 – Security, Observability, and DoS Prevention

This module wraps the entire system in a production-grade security and monitoring layer, directly satisfying **Guideline #28** (Database Security) and **Guideline #29** (Performance Analysis).

- **SQL Injection Firewall:** A FastAPI middleware proxy parses every incoming SQL query into an **Abstract Syntax Tree (AST)** using the `sqlglot` library. Queries containing banned patterns (e.g., `DROP`, `UNION SELECT`, `--` comment injections) are rejected with HTTP 403 and logged to a `security_events` table.

- **DoS Rate Limiter:** Each client is fingerprinted by IP. A Redis-backed **sliding window counter** enforces a per-client QPS ceiling. Clients exceeding the limit are throttled. A Siege DDoS simulation will demonstrate the system remaining stable under attack.
- **Grafana Dashboard:** Five live panels connected to TimescaleDB: (a) candlestick chart, (b) order book depth heatmap, (c) volume bars, (d) quantum arbitrage signal feed, (e) system QPS and 99th-percentile latency.
- **Prometheus Metrics:** DBMS-internal statistics (query counts, cache hit ratio, lock wait times) and OS-level metrics (CPU, RAM, Disk I/O) are scraped by Prometheus and displayed in Grafana.

Technology Stack

Component	Technology
Data Ingestion	Apache Kafka, WebSocket (Binance / Alpha Vantage free-tier API)
Matching Engine	Python 3.12 (or Java 21), Red-Black Tree, LMAX Disruptor Ring Buffer
Temporal Database	TimescaleDB 2.x (PostgreSQL 16 extension)
Graph Database	Apache AGE (PostgreSQL extension), Cypher query language
Quantum Engine	IBM Qiskit 1.x (local QASM simulator), Python 3.12
Security Proxy	FastAPI, `sqlglot` AST parser, Redis 7
Monitoring	Grafana 10, Prometheus, TimescaleDB data source
Benchmarking	Siege 4.x, custom Python ThreadPool + ConnectionPool
Frontend (Optional)	Next.js 15 for live dashboard UI
Version Control	Git (GitHub), `dev` → `main` branching policy

Weekly Work Plan

Timeline	Milestone & Deliverables
Week 1 **Mar 8 – Mar 14**	**Infrastructure Setup & Proposal Submission** • Submit this proposal for instructor approval • Set up PostgreSQL 16 + TimescaleDB extension on local server • Install Apache AGE graph extension and test Cypher connectivity • Set up Apache Kafka broker and create `raw_orders` topic • Set up IBM Qiskit environment (local QASM simulator) • Define and freeze all database schemas (tick data, order book, graph, arbitrage signals, security events) • Create GitHub repository with `dev/main` branching policy

Timeline	Milestone & Deliverables (Continued)
Week 2 **Mar 15 – Mar 21**	**Core Engine Implementation** • Build Red-Black Tree + FIFO linked list LOB data structure (Member 1) • Implement LMAX Disruptor ring buffer and 3-thread pipeline (Member 1) • Implement Kafka consumer → TimescaleDB bulk `COPY` pipeline (Member 2) • Create hypertables and define retention/compression policies (Member 2) • Implement and test Bellman-Ford classical arbitrage baseline (Member 3)
Week 3 **Mar 22 – Mar 28**	**Intermediate Evaluation Preparation** • Build continuous aggregates (1m, 5m, 15m, 1h OHLCV) (Member 2) • Implement SQL window function indicators: VWAP, SMA, Bollinger Bands (Member 2) • Build Apache AGE graph population script and 500ms edge-weight update worker (Member 3) • Design and test Grover Oracle circuit for 8-node currency graph (Member 4) • Begin FastAPI SQL injection proxy with `sqlglot` AST parser (Member 5) • **Intermediate Evaluation: demo LOB engine + TimescaleDB pipeline**
Week 4 **Mar 29 – Apr 7**	**Quantum Module and Security Integration** • Scale Grover circuit to 16, 32, 64 currency nodes (Member 4) • Integrate Qiskit middleware with PostgreSQL `arbitrage_signals` table (Member 4) • Run quantum vs. Bellman-Ford benchmarking and generate plots (Member 4) • Complete Redis-backed DoS rate limiter and Siege DDoS simulation (Member 5) • Set up Prometheus scraping + Grafana dashboard with all 5 panels (Member 5) • Full end-to-end integration test of all 5 modules (all members)

Timeline	Milestone & Deliverables (Continued)
Week 5 **Apr 8 – Apr 14**	**Benchmarking, Report & Demo Preparation** • Run full Siege QPS benchmarking (target: >100k orders/sec baseline) • Collect all screenshots for report (Grafana panels, query outputs, circuit diagrams) • Write complete 10-page final report (all members contribute sections) • Prepare live demonstration script covering all 5 modules • Code freeze, final review, and submission
April 15	**Final Submission & Live Demonstration**

Expected Deliverables and Outcomes

1. **Working LOB Matching Engine:** REST and WebSocket API handling >100,000 order operations per second, benchmarked with Siege.
2. **TimescaleDB Analytics Layer:** Hypertable storing ≥1 million synthetic tick records with automated OHLCV aggregates provably querying 100x faster than standard PostgreSQL.
3. **Live Currency Graph:** Apache AGE graph with ≥20 nodes and real-time edge weight updates, supporting Cypher path queries.
4. **Quantum Arbitrage Engine:** Grover circuit successfully detecting a profitable 3-hop cycle from a 16-node currency graph, with a benchmarking report showing $O(\sqrt{N})$ vs $O(N)$ scaling.
5. **Security & Monitoring:** SQL injection firewall blocking standard OWASP SQL injection payloads, DoS rate limiter demonstrated under Siege load, and a live Grafana dashboard.
6. **10-Page Final Report:** Covering all five modules with methodology, architecture diagrams, SQL listings, circuit diagrams, and benchmark results.

References

[1] M. Thompson et al., *Disruptor: High performance alternative to bounded queues for exchanging data between concurrent threads*, LMAX Exchange, 2011.

[2] M. Fowler, *The LMAX Architecture*, martinfowler.com, July 2011.

[3] TimescaleDB Documentation, *Analyze financial tick data with TimescaleDB.*

[4] TraderMade, *TimescaleDB Tutorial: Real-Time Market Data to OHLC Candles*, 2021.

[5] Classiq, *Grover's Algorithm – Theory and Implementation*, 2024.

[6] L. K. Grover, *A fast quantum mechanical algorithm for database search*, Proceedings of 28th ACM Symposium on Theory of Computing, 1996.

[7] Apache Software Foundation, *Apache AGE – Graph Extension for PostgreSQL.*

[8] IBM Quantum, *Quantum Approximate Optimization Algorithm (QAOA)*, IBM Quantum Documentation, 2025.

[9] J. Postel et al., *Siege – HTTP Load Testing and Benchmarking Utility.*

[10] PostgreSQL Global Development Group, *PostgreSQL 16 Documentation.*